LAPORAN PRAKTIKUM

Pemrograman Berorientasi Objek

Ekspansi Produk Keuangan (Inheritance)

disusun Oleh:

Nama: RIFKI MAULANA

NIM: 2511533007

Kelas: A

DOSEN PENGAMPU: NURFIAH, S.ST, M.Kom

ASISTEN PRAKTIKUM: M. DAWI SYAUQI



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 03 Oktober 2026

Laprak 1

A. Langkah Kerja Praktikum

a. Membuat class Rekening

```
public class Rekening {
```

1. Buat class Rekening

   - Class sebagai blueprint dari sebuah objek

```
String nomorRekening;
String namaPemilik;
double saldo;
```

2. Tambahkan state/atribut

   - Setiap objek memiliki state atau kondisi yang independent. Berfungsi sebagai identitas dan penyimpanan data dari sebuah objek

```
public Rekening(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
}
```

3. Buat constructor

   - Constructor adalah blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat menggunakan keyword new. Fungsinya sebagai pemberi nilai awal pada saat sebuah objek pertama kali dibuat. Nanti akan ada objek objek yang dikirim ke nomor, nama, dan saldoAwal yang kemudian dimasukkan ke variable yang telah ditentukan yaitu nomorRekening, namaPemilik, dan saldo

4. Buat method yang diperlukan

```
public void setorTunai(double nominal) {
    if (nominal >= 10000) {
        saldo += nominal;
        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp " + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 10000!");
    }
}
```

- Method ini berfungsi menyetorkan tunai ke akun dengan catatan bahwa nominal yang disetorkan harus lebih atau sama dengan Rp10.000 dan kemudian nominal ditambahkan ke variable saldo

```
public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik :" + namaPemilik);
    System.out.println("Saldo Akhir  : Rp" + saldo);
    System.out.println("----------------------");
}
```

- Method ini bertugas untuk menampilkan informasi rekeningseperti nomorRekening, namaPemilik, dan saldo

```
package Pekan1;

public class Rekening {

    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal) {
            nomorRekening = nomor;
            namaPemilik = nama;
            saldo = saldoAwal;
            System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan
saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
            if (nominal >= 10000) {
                    saldo += nominal;
                    System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini:
Rp " + saldo);
            } else {
                    System.out.println("Gagal: Nominal setor harus lebih dari 10000!");
            }
    }

    public void cekInformasi() {
            System.out.println("--- INFO REKENING ---");
            System.out.println("No. Rekening : " + nomorRekening);
            System.out.println("Nama Pemilik :" + namaPemilik);
            System.out.println("Saldo Akhir  : Rp" + saldo);
            System.out.println("---------------------- ");
    }

    public void tarikTunai(double nominal) {
            if (nominal < 10000) {
                    System.out.println("Transaksi Gagal : minimal nominal penarikan 10.000");
            } else if (nominal > saldo) {
                    System.out.println("Transaksi Gagal: saldo tidak mencukupi. Saldo Anda: Rp"
+ saldo);
            } else {
                    saldo = saldo - nominal;
                    System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini:
Rp" + saldo);
            }
    }

}
```

b. Membuat class Main

1. Buat scanner terlebih dahulu

```
package Pekan1;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
```

2. Menginisialisasi scanner untuk input user nantinya

```
Rekening akunAktif = null; //Objek belum diinisialisasi (null)
boolean isRunning = true;
```

   - Rekening akunAktif = null; digunakan untuk menyimpan data rekening, diberikan nilai null di awal karena saat kode dijalankan, user belum membuka rekening
   - Boolean isRunning = true; selama nilainya ture, menu pilihan akan terus muncul sampai user memilih angka 0

3. Buatkan menu pilihan yang akan dipilih oleh user

```
System.out.println("=== SISTEM PERBANKAN MINI ===");

while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("0. Keluar");
    System.out.print("Pilih menu: ");

    int pilihan = input.nextInt();
    input.nextLine(); // Membersihkan buffer enter
```

   - Untuk menampilkan kepada user apa saja yang bisa dilakukan pada menu

4. Buat switch case pilihan

```
switch (pilihan) {
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    if (saldo < 50000) {
        System.out.println("saldo kurang");
    } else {
        akunAktif = new Rekening(no, nama, saldo);
        daftarRekening.add(akunAktif);
        break;
    }
```

- Case 1 untuk membuka rekening, ia memasukkan no, nama, dan saldo kedalam Rekening

```
case 2:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan nominal setor: ");
        double setor = input.nextDouble();
        akunAktif.setorTunai(setor); // Memanggil Behavior / method
    }
    break;
```

- Case 2 berfungsi untuk melakukan setor tunai menggunakan method setorTunai yang ada pada Rekening
- Case 3 adalah tugas

```
case 4:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        akunAktif.cekInformasi();
    }
    break;
```

- Case 4 berfungsi untuk menampilkan informasi dari rekening menggunakan method cekInformasi
- Case 5 adalah tugas

```
    case 0:
        isRunning = false;
            System.out.println("Sistem ditutup. Terima Kasih!");
        break;

    default:
        System.out.println("Pilihan tidak valid!");
    }

}
input.close();
```

- Case 0 berguna untuk menghentikan sistem atau program
- Default berfungsi jika pilihan tidak valid
- Jangan lupa close input dari program

```java
package Pekan1;
import java.util.Scanner;
import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {
            Scanner input = new Scanner(System.in);
            ArrayList<Rekening> daftarRekening = new ArrayList();
            Rekening akunAktif = null; //Objek belum diinisialisasi (null)
            boolean isRunning = true;

            System.out.println("=== SISTEM PERBANKAN MINI ===");

            while (isRunning) {
                    System.out.println("\nMenu Utama:");
                    System.out.println("1. Buka Rekening Baru");
                    System.out.println("2. Setor Tunai");
                    System.out.println("3. Tarik Tunai");
                    System.out.println("4. Cek Informasi Rekening");
                    System.out.println("5. Ganti Akun");
                    System.out.println("0. Keluar");
                    System.out.print("Pilih menu: ");

                    int pilihan = input.nextInt();
                    input.nextLine(); // Membersihkan buffer enter

                    switch (pilihan) {
                    case 1:
                            System.out.print("Masukkan No Rekening: ");
                            String no = input.nextLine();
                            System.out.print("Masukkan Nama Pemilik: ");
                            String nama = input.nextLine();
                            System.out.print("Masukkan Saldo Awal: ");
                            double saldo = input.nextDouble();
                            if (saldo < 50000) {
                                    System.out.println("saldo kurang");
                            } else {
                                    akunAktif = new Rekening(no, nama, saldo);
                                    daftarRekening.add(akunAktif);
                                    break;
                            }

                    case 2:
```

```
                                if (akunAktif == null) {
                                        System.out.println("Error: Mohon maaf, Anda belum memiliki
nomor rekening!");
                                } else {
                                        System.out.print("Masukkan nominal setor: ");
                                        double setor = input.nextDouble();
                                        akunAktif.setorTunai(setor); // Memanggil Behavior / method
                                }
                                break;

                        case 3:
                                if (akunAktif == null) {
                                        System.out.println("Error: Anda belum memiliki nomor
rekening");
                                } else {
                                        System.out.println("Masukkan nominal tarik tunai: ");
                                        double tarik = input.nextDouble();
                                        akunAktif.tarikTunai(tarik);
                                }
                                break;

                        case 4:
                                if (akunAktif == null) {
                                        System.out.println("Error: Anda belum membuka rekening!");
                                } else {
                                        akunAktif.cekInformasi();
                                }
                                break;

                        case 5:
                                System.out.println("Masukkan No Rekening Akun yang dicari:");
                                String norek = input.nextLine();
                                boolean ditemukan = false;

                                for (Rekening rek : daftarRekening) {
                                        if (rek.nomorRekening.equals(norek)) {
                                                akunAktif = rek;
                                                ditemukan = true;
                                                System.out.println("Berhasil Akun aktif saat ini
atas nama: " + rek.namaPemilik);
                                                break;
                                        }
                                }

                                if (!ditemukan) {
                                        System.out.println("Error: Nomor rekening tidak ditemukan");
                                }
                                break;

                        case 0:
                                isRunning = false;
                                        System.out.println("Sistem ditutup. Terima Kasih!");
                                break;


                        default:
                                System.out.println("Pilihan tidak valid!");
                        }

                }
                input.close();

        }

}
```

B. Tugas/Latihan praktikum

1. Implementasi Fitur Tarik tunai

a. Tambahkan method tarikTunai di Rekening

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : minimal nominal penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
    } else {
        saldo = saldo - nominal;
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    }
}
```

Mehod memeriksa apakah nominal < 10000 dan apakah nominal > saldo, yang mana nominal adalah jumlah saldo yang ingin di ambil user, jika kedua if tidak memenuhi maka user baru bisa Tarik tunai yang mana akan mengurangi saldonya sebanyak nominal yang ditarik

b. Panggil method di case 3

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memiliki nomor rekening");
    } else {
        System.out.println("Masukkan nominal tarik tunai: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;
```

Program akan mengecek apakah akunAktif, kemudian method tarikTunai dipanggil

2. Tantangan Multi-Akun

a. Import dan deklarasikan arraylist

```
import java.util.ArrayList;
```

```
ArrayList<Rekening> daftarRekening = new ArrayList();
```

b. Ubah case 1pada bagian else {}

```
else {
  akunAktif = new Rekening(no, nama, saldo);
  daftarRekening.add(akunAktif);
  break;
```

- Diubah karena kita ingin memasukkan data akunAktif ke array

c. Tambahkan 5. Ganti akun ke list menu pilihan

```
System.out.println("5. Ganti Akun");
```

d. Tambahkan case 5

```
case 5:
    System.out.println("Masukkan No Rekening Akun yang dicari:");
    String norek = input.nextLine();
    boolean ditemukan = false;

    for (Rekening rek : daftarRekening) {
        if (rek.nomorRekening.equals(norek)) {
            akunAktif = rek;
            ditemukan = true;
            System.out.println("Berhasil Akun aktif saat ini atas nama: " + rek.namaPemilik);
            break;
        }
    }

    if (!ditemukan) {
        System.out.println("Error: Nomor rekening tidak ditemukan");
    }
```

- Deklarasikan dulu bahwa ditemukan = false
- Cari untuk setiap Rekening yang ada pada daftar Rekening, jika nomorRekeningnya sama dengan input dari user maka Rekening diganti dengan inputan yang dibuat oleh user, kemudian nilai dari ditemukan diubah menjadi true
- Jika perulangan for tidak menghasilkan apa apa maka if(!ditemukan) bisa jalan karena pada awal pembuatan variable ditemukan sudah ditentukan bahwa ditemukan = false sehingga nilai dari if(!ditemukan) itu true sehingga bisa dijalankan. Jika saja for menghasilkan sesuatu maka ditemukan langsung diubah jadi true sehingga if(!ditemukan) tidak bisa dijalankan

Laprak 2

A. Langkah Kerja Praktikum

a. Membuat class Diagram Transaksi

1. Buat class Transaksi

```
public class Transaksi {
```

- Class sebagai blueprint dari sebuah objek

2. Buatkan attribute dan constructor

```
String idTransaksi;
String jenis;
double nominal;

// constructor
public Transaksi(String id, String jenis, double nominal){
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

- Setiap objek memiliki state atau kondisi yang independent. Berfungsi sebagai identitas dan penyimpanan data dari sebuah objek
- Constructor adalah blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat menggunakan keyword new. Fungsinya sebagai pemberi nilai awal pada saat sebuah objek pertama kali dibuat. Nanti akan ada objek objek yang dikirim ke nomor, nama, dan saldoAwal yang kemudian dimasukkan ke variable yang telah ditentukan yaitu nomorRekening, namaPemilik, dan saldo.

3. Buat method yang diperlukan

```
public void cetakDetail() {
    System.out.println("ID: " + idTransaksi + " | jenis: " + jenis + " | Nominal: Rp" + nominal);
}
```

- Method ini berfungsi untuk menccetak detail sebuah transaksi dalam satu baris, yaitu menampilkan idTransaksi, jenis, dan nominal.

4. Buka Kembali file Rekening.java dan tambahkan arraylist

```java
// Implementasi Asosiasi (1-to-many)
ArrayList<Transaksi> riwayatTransaksi;

public Rekening(String nomor, String nama, double saldoAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;

    // Wajib menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
    this.riwayatTransaksi = new ArrayList<>();

    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
}
```

- Arraylist berfungsi untuk menyimpan akun akun yang dibuat

5. Catat Riwayat transaksi

```java
public void setorTunai(double nominal) {
    if (nominal >= 0) {
        saldo += nominal;
        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp " + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}
```

- currentTimeMillis digunakan untuk mendapatkan waktu saat ini atau saat transaksi dibuat
- method ini menambah saldo rekening sekaligus mencatat transaksi ke riwayatTransaksi

```java
package Pekan2;

public class Transaksi {

    String idTransaksi;
    String jenis;
    double nominal;

    // constructor
    public Transaksi(String id, String jenis, double nominal){
            this.idTransaksi = id;
            this.jenis = jenis;
            this.nominal = nominal;
    }
    public void cetakDetail() {
            System.out.println("ID: " + idTransaksi + " | jenis: " + jenis + " |
Nominal: Rp" + nominal);
    }

}
```

```java
package Pekan2;
```

```
-    import java.util.ArrayList;
-
-    public class Rekening {
-
-        String nomorRekening;
-        String namaPemilik;
-        double saldo;
-
-        // Implementasi Asosiasi (1-to-many)
-        ArrayList<Transaksi> riwayatTransaksi;
-
-        public Rekening(String nomor, String nama, double saldoAwal) {
-                this.nomorRekening = nomor;
-                this.namaPemilik = nama;
-                this.saldo = saldoAwal;
-
-                // Wajib menginisialisasi ArrayList di dalam constructor agar tidak
    NullPointerException
-                this.riwayatTransaksi = new ArrayList<>();
-
-                System.out.println("Rekening atas nama " + namaPemilik + " berhasil
    dibuat dengan saldo Rp" + saldo);
-        }
-
-        public void setorTunai(double nominal) {
-                if (nominal >= 0) {
-                        saldo += nominal;
-                        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
-                        String idTrx = "TRX-S-" + System.currentTimeMillis();
-                        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
-                        riwayatTransaksi.add(trxBaru);
-
-                        System.out.println("Setor tunai Rp" + nominal + " berhasil.
    Saldo saat ini: Rp " + saldo);
-                } else {
-                        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
-                }
-        }
-
-        public void cekInformasi() {
-                System.out.println("--- INFO REKENING ---");
-                System.out.println("No. Rekening : " + nomorRekening);
-                System.out.println("Nama Pemilik :" + namaPemilik);
-                System.out.println("Saldo Akhir  : Rp" + saldo);
-                System.out.println("----------------------");
-        }
-
-        public void tarikTunai(double nominal) {
-                if (nominal < 10000) {
-                        System.out.println("Transaksi Gagal : minimal nominal penarikan
    10.000");
-                } else if (nominal > saldo) {
-                        System.out.println("Transaksi Gagal: saldo tidak mencukupi.
    Saldo Anda: Rp" + saldo);
-                } else {
-                        saldo = saldo - nominal;
-                        String idTrx = "TRX-T-" + System.currentTimeMillis();
-                        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
-                        riwayatTransaksi.add(trxBaru);
-                        System.out.println("Tarik tunai Rp" + nominal + " berhasil.
    Saldo saat ini: Rp" + saldo);
-                }
-        }
-        public void cetakMutasi() {
-                System.out.println("--- Mutasi Rekening " + nomorRekening + " ---");
-                if (riwayatTransaksi.isEmpty()) {
-                        System.out.println("Tidak ada transaksi pada rekening ini");
```

```
-               } else {
-                       for (Transaksi transaksi : riwayatTransaksi) {
-                               transaksi.cetakDetail();
-                       }
-
-               }
-
-       }
-
-   }
```

B. Tugas/Latihan praktikum

1. Integrasi Rekam Jejak Penarikan

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : minimal nominal penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
    } else {
        saldo = saldo - nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    }
}
```

2. Fitur Cetak Mutasi Rekening

   a. Buat method cetakMutasi()

```
public void cetakMutasi() {
    System.out.println("--- Mutasi Rekening " + nomorRekening + " ---");
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Tidak ada transaksi pada rekening ini");
    } else {
        for (Transaksi transaksi : riwayatTransaksi) {
            transaksi.cetakDetail();
        }

    }

}
```

- Method ini berfungsi untuk mencetak mutasi dari rekening. Pertama tama dicek dulu apakah punya riwayatTransaksi kosong atau tidak, kemudian jika tidak kosong maka untuk setiap transaksi di riwayatTransaksi, lakukan method cetakDetail

b. Tambahkan menu baru 6. Cetak Mutasi (Riwayat)

```
System.out.println("6. Cetak Mutasi (Riwayat");
```

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memiliki nomor rekening");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

- Switch case akan melihat terlebih dahulu apakah akunAktif kemudian jika nilainya tidak null maka barulah memanggil method cetakMutasi()

Laprak 3

A. Langkah Kerja Praktikum

1. Langkah 1 Mengunci entitas transaksi

   Pada file tranksaksi.java ubah variable menjadi private dan berikan getter untuk data yang perlu dibaca. Objek transaksi bersifat immutable (tidak boleh diubah setelah dibuat. sehingga tidak perlu setter

```java
package Pekan3;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // constructor
    public Transaksi(String id, String jenis, double nominal){
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya menyediakan Getter (Read-Only)
    public String getIdTransaksi() { return idTransaksi;}
    public String getJenis() { return jenis;}
    public double getNominal() { return nominal;}

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | jenis: " + jenis + " | Nominal: Rp" + nominal);
    }

}
```

- Buat var idTransaksi, jenis dan nominal jadi private
- Berikan getter untuk membaca var

2. Langkah 2: Mengunci entitas rekening dan menambahkan PIN

   Pada file rekening.java amankan saldo dan tambahkan atribut PIN

```java
import java.util.ArrayList;

public class Rekening {
    // 1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data sensitif


    // Implementasi Asosiasi (1-to-many)
    private ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening(String nomor, String nama, String pinAwal, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;
        // Validasi PIN di dalam Constructor
        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }
        // Wajib menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening;}
    public String getNamaPemilik() { return namaPemilik;}

    // 4. Method otentikasi internal (Validasi Enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }
```

- Buat nomorRekening, namaPemilik, saldo, riwayatTransaksi menjadi private dan tambahkan private pin

- Modif constructor untuk menerima pin dengan catatan bahwa pin harus berjumlah 6 karakter dan jika tidak menginputkan 6 karakter maka akna menggunakan pin default
- Buat getter untuk atribut yang boleh dibaca oleh public yaitu untuk nomorRekening dan namaPemilik
- Buat method untuk validasi pin(validasi enkapsulasi)

  Mengapa tidak membuat setSaldo?

  = karena nantinya pengguna bisa mengubah saldo, saldo tidak boleh diubah oleh pengguna

  Mengapa tidak membuat getPin?

  = tujuan dari pembuatan pin adalah untuk menjaga akun pengguna dari orang lain, jika dibuat getPin maka pin bisa dilihat

B. Tugas/Latihan praktikum

Tugas : Mengamankan CLI dengan Otentikasi

1. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan PIN(6 digit angka): ");
    String pin = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();

    if (saldo < 50000) {
        System.out.println("saldo kurang");
    } else {
        akunAktif = new Rekening(no, nama, saldo, pin);
        daftarRekening.add(akunAktif);
    }
    break;
```

- Tambahkan inputan pin dan tambahkan pin pada parameter akunAktif

2. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.
    - Panggil fungsi akunAktif.otentikasi(pinYangDiinput).
    - Jika mengembalikan nilai true, jalankan transaksi.
    - Jika false, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memiliki nomor rekening");
    } else {
        System.out.println("Masukkan PIN: ");
        String inputPin = input.nextLine();
        if (akunAktif.otentikasi(inputPin) == true) {
            System.out.println("Masukkan nominal tarik tunai: ");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses DItolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memiliki nomor rekening");
    } else {
        System.out.println("Masukkan PIN: ");
        String inputPin = input.nextLine();
        if (akunAktif.otentikasi(inputPin) == true) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

- Program melakukan if untuk melihat apakah method otentikasi dengan argument inputPin bernilai true
- Jika tidak maka tidak akan menjalankan method yang diharapkan(tarikTunai dan cetakMutasi)

Laprak 4

A. Langkah Kerja Praktikum

3. Langkah 1: Memastikan Akses Modifier di Rekening.java

   Mengubah atribut saldo dan arraylist transaksi menjadi protected supaya yang bisa mengakses hanyalah class itu, class lain dalam package yang sama, dan subclass(baik dari package yang sama maupun dari package yang lain.

```java
public class Rekening {
    // 1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private String pin; // Data sensitif

    // Gunakan protected agar subclass bisa mengaksesnya langsung
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;
```

4. Langkah 2: Membuat Subclass RekeningTabungan

   Disinilah kita menggunakan inheritance, membuat subclass dari class Rekening. Inheritance bisa mengambil atribut dari class parent(jika memungkinkan) maupun method dari class parent.

```java
public class RekeningTabungan extends Rekening{

    // Atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    // Constructor  Subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
        // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        // Menghitung bunga
        // mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan?
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        // Mencatat riwayat transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }

}
```

- Buat atribut baru di subclass RekeningTabungan yaitu atribut sukuBunga
- Tambahkan method baru tambahBungaAkhirBulan yang berfungsi untuk menambahkan saldo dalam persen sesuai inputan

5. Langkah 3: Membuat Subclass RekeningGiro

   Buat RekeningGiro yang berguna untuk limit pinjaman

```
public class RekeningGiro extends Rekening{
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft )
        // Memanggil inisialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
    // penarikan hingga limit overdraft diselesaikan di modul 5
}
```

   - Kita menggunakan getter untuk mengembalikan batasOverdraft

B. Tugas/Latihan praktikum

3. Pembuatan Multi-Produk

   Pada file main.java, tepatnya di case 1 buatlah:

   a. Setelah user menginputkan Saldo awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"

   b. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.

   c. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.

   d. Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan PIN(6 digit angka): ");
    String pin = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    System.out.println("Pilih Produk(1/2): \n1. Tabungan Umum \n2. Giro Bisnis");
    int pilProduk = input.nextInt();

    if (saldo >= 50000) {
        if (pilProduk == 1) {
            System.out.println("Masukkan suku bunga(dalam persen): ");
            double sukuBunga = input.nextDouble();
            akunAktif = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
        }
        else if (pilProduk == 2) {
            System.out.println("Masukkan batas overdraft(limit pinjaman): ");
            double batasOverdraft = input.nextDouble();
            akunAktif = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
        }
        else {
            System.out.println("pilih antara 1 dan 2");
        }
    }
    else {
        System.out.println("saldo tidak boleh kurang dari 50.000");
    }
    break;
```

- Buat input baru berupa pilihan antara angka 1 atau dua untuk Tabungan umum dan giro bisnis, jika tidak program akan menyuruh untuk menginputkan angka 1 atau 2
- Jika input 1 program meminta input sukuBunga, kemudian mengaktifkan akun RekeningTabungan
- Jika input 2 program meminta input baatsOverdraft, kemudian mengaktifkan akun RekeningGiro

4. Simulasi Waktu Akhir Bulan
   a. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
   b. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof
   c. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
   d. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

```
case 7:
    if (akunAktif != null) {
        if (akunAktif instanceof RekeningTabungan) {
            RekeningTabungan rek = (RekeningTabungan) akunAktif;
            rek.tambahBungaAkhirBulan();
        }
        else {
            System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
        }
    }
    else {
        System.out.println("Akun belum aktif");
    }
    break;
```

- Program cek dulu apakah akunAktif, kemudian baru men cek apakah akunAktif saat ini mengggunakan instanceof, jika yam aka jalankan method tambahBungaAkhirBulan yang ada di subclass RekeningTabungan.
- Jika tidak ada akunAktif maka program menampilkan akun belum aktif
- Jika tidak ada akunAktif untuk RekeningTabungan maka program menyatakan gagal

Laprak 5

A. Langkah Kerja Praktikum

1. Langkah 1: Refaktor Method tarikTunai pada Kelas Induk (Rekening.java)

Pada praktikum sebelumnya, RekeningGiro sudah dibuat, tetapi logika tarikTunai-nya masih mengikuti rekening biasa, yaitu saldo tidak boleh minus. Agar penarikan dapat dipakai sebagai bagian dari fitur transfer nanti, tipe kembalian (return type) method tarikTunai pada Rekening.java diubah dari void menjadi boolean. Method mengembalikan true jika penarikan berhasil dan false jika gagal.

```
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Transaksi Gagal : minimal nominal penarikan 0");
        return false;
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        return false;
    } else {
        saldo = saldo - nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        return true;
    }
}
```

- Setiap cabang sekarang mengembalikan nilai: false untuk transaksi yang gagal dan true untuk transaksi yang sukses.
- Nilai kembalian ini nantinya dipakai oleh method transfer untuk memutuskan apakah dana boleh disetor ke rekening tujuan.

2. Langkah 2: Melakukan Overriding pada RekeningGiro.java

Selanjutnya, logika tarikTunai ditimpa (override) di RekeningGiro agar memakai aturan batasOverdraft. Penarikan diperbolehkan selama nominal tidak melebihi saldo ditambah batasOverdraft, sehingga saldo boleh bernilai negatif.

```java
// implementasi polimorfisme overriding
@Override
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
        return false;
    }
    // Logika khusus giro: Boleh ditarik asalkan nominal tidak melebihi (saldo + batasOverdraft)
    else if (nominal > (saldo + batasOverdraft)) {
        System.out.println("Transaksi Gagal: melebihi batas Overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
        return false;
    }
    else {
        saldo -= nominal; // saldo bisa bernilai negatif disini (memakan overdraft)

        String idTrx = "TRX-G-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

        System.out.println("Tarik tunai giro Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        return true;
    }
}
```

- Anotasi @Override memberi tahu compiler bahwa method ini sengaja menimpa method milik kelas induk. Jika terjadi salah ketik pada nama method, compiler akan menampilkan pesan error.
- Atribut saldo dan riwayatTransaksi bisa diakses langsung karena dideklarasikan protected di kelas Rekening.

3. Langkah 3: Uji Coba Pengikatan Dinamis (Dynamic Method Dispatch)

Program Main.java dijalankan tanpa mengubah satu baris pun kode pada Menu 3 (Tarik Tunai). Dibuat akun baru dengan Produk 2 (Giro Bisnis), saldo awal 100.000 dan batas overdraft 500.000, lalu dilakukan tarik tunai sebesar 400.000.

Pada rekening biasa, penarikan ini pasti ditolak karena melebihi saldo. Namun karena objek yang dibuat adalah RekeningGiro, JVM menentukan saat program berjalan bahwa method tarikTunai yang dipanggil adalah versi milik RekeningGiro. Hasilnya penarikan berhasil dan saldo menjadi minus 300.000. Inilah yang disebut Dynamic Method Dispatch (late binding): Menu 3 hanya memanggil akunAktif.tarikTunai(), tanpa perlu if-else untuk memeriksa tipe rekening.

B. Tugas/Latihan Praktikum

1. Membuat Method Transfer pada Rekening.java

Pada class Rekening ditambahkan satu method baru dengan signature public void transfer(Rekening tujuan, double nominal, String pinInput).

```java
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("Transfer dibatalkan: PIN yang anda masukkan salah!");
        return;
    }

    boolean berhasil = this.tarikTunai(nominal);

    if (berhasil) {
        tujuan.setorTunai(nominal);
        System.out.println("Transfer Rp" + nominal + " ke " + tujuan.getNamaPemilik() + " berhasil");
    }
}
```

- Parameter Rekening tujuan bersifat polimorfik, sehingga method ini dapat menerima objek RekeningTabungan maupun RekeningGiro sebagai tujuan.
- Langkah pertama memanggil otentikasi(pinInput). Jika hasilnya false, transfer dibatalkan dan method berhenti.
- Langkah kedua memanggil this.tarikTunai(nominal). Karena method ini di-override, jika pengirim adalah RekeningGiro maka aturan overdraft yang dipakai, sedangkan jika pengirim adalah RekeningTabungan maka aturan penarikan biasa yang dipakai.
- Langkah ketiga memeriksa nilai kembalian tarikTunai. Jika true, method memanggil tujuan.setorTunai(nominal). Jika false, tidak ada dana yang berpindah.

2. Integrasi ke CLI Utama (Main.java)

Pada Main.java ditambahkan menu baru "8. Transfer ke Rekening Lain". Baris menu ditambahkan sebelum "0. Keluar":

Lalu ditambahkan case 8 pada switch:

```java
case 8:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memiliki nomor rekening");
    } else {
        System.out.print("Masukkan No Rekening Tujuan: ");
        String noTujuan = input.nextLine();
        Rekening akunTujuan = null;

        for (Rekening rek : daftarRekening) {
            if (rek.getNomorRekening().equals(noTujuan)) {
                akunTujuan = rek;
                break;
            }
        }

        if (akunTujuan == null) {
            System.out.println("Error: Nomor rekening tujuan tidak ditemukan");
        } else {
            System.out.print("Masukkan nominal transfer: ");
            double nominalTransfer = input.nextDouble();
            input.nextLine(); // bersihkan buffer enter sebelum membaca String
            System.out.print("Masukkan PIN: ");
            String pinTransfer = input.nextLine();

            akunAktif.transfer(akunTujuan, nominalTransfer, pinTransfer);
        }
    }
    break;
```

- Program memeriksa terlebih dahulu apakah akunAktif sudah ada. Jika belum, program menampilkan pesan error.
- Pengguna memasukkan nomor rekening tujuan, lalu program mencari nomor tersebut di dalam daftarRekening. Jika tidak ditemukan, program menampilkan pesan error.
- Jika ditemukan, pengguna memasukkan nominal transfer dan PIN, kemudian program memanggil akunAktif.transfer(akunTujuan, nominalTransfer, pinTransfer).
- Agar pencarian dapat menemukan rekening, objek yang baru dibuat pada case 1 perlu disimpan ke daftarRekening dengan perintah daftarRekening.add(akunAktif); tepat setelah objek RekeningTabungan maupun RekeningGiro dibuat (upcasting: subclass otomatis dikenali sebagai tipe Rekening).